Dela Cruz, Cloie Sofia R.
Morales, Rayleen Claire G.
AM1

Unveiling Hidden Patterns and Optimal Strategies with Machine Learning

A. Data Acquisition & Preprocessing

Data Source Research

We chose the Brazilian E-Commerce Public Dataset by Olist because it provides a detailed and abundant collection of real-world transaction data from an online marketplace. It contains many features necessary for our project like customer orders, order items, product information, and customer characteristics, such as purchasing frequency, spending behavior, and product preferences. This makes the dataset highly relevant for our project's goals of customer segmentation and association rule mining.

Strengths

1. **Real-world scale**
    a. The dataset covers over ninety thousand transactions over three years from a real online store. This provides enough data for reliable clustering and association analysis.
2. **Rich attributes and features**
    a. This dataset includes order-level, item-level, and product category information. This allows for both behavioral segments through KMeans and market basket analysis through Apriori.
3. **Structured and well-documented**
    a. The dataset is relatively clean, public, and cited in many researches. The creators of this dataset also provided detailed documentation in the Kaggle post that helped us in analysing the dataset. These make it very easy to work with and validate against existing benchmarks.

Weaknesses

1. **High proportion of one-time purchases**
    a. During our initial exploration of the dataset, we noticed that most customers purchased only once. This can limit the insights we can gather into long-term customer behavior.
2. **Sparse multi-item transactions**
    a. We also noticed that many orders contain only one item. This can be a potential bottleneck for our Apriori algorithm because it reduces the strength of association rules generated from co-purchase behavior.
3. **Missing product attributes**

a. Some products in the dataset (like dimension and English category names) also had missing values. This required additional preprocessing and imputation steps for us.

Data Loading & Initial Exploration

For initial data exploration, we create a helper function named explore_dataframe(). This function helped us automate the process of quickly running the necessary methods and processes to summarize the structure and quality of our datasets. It outputs the following information:

- Shape of the dataset to understand dataset size with .head()
- First 5 rows for a quick look into the data format and sample values with .info()
- Info summary of columns including non-null counts and data types
- Statistical summary of numerical columns. This is useful for spotting outliers and understanding the distribution of the data
- Missing values count for each column to identify data quality issues
- Duplicate rows count, and
- Data types for each column to confirm that numeric values are not stored as strings

The explore_dataframe() function improved efficiency by providing a standardized view of each dataset, reducing repetitive manual inspection steps.

Exploration Results

1. **orders DataFrame (olist_orders_dataset.csv)**
   The olist_orders_dataset.csv file contains 99,441 rows and 8 columns, each representing a unique order placed on the Olist platform. The first few rows show details like order_id, customer_id, order_status, and several timestamps related to order creation, approval, shipping, and delivery. This dataset primarily focuses on order-level attributes and does not include individual product information.

   From the .info() summary, all columns are stored as object types (strings). The dataset has no duplicate rows. However, there are missing values in three columns:
   - order_approved_at = 160 missing values
   - order_delivered_carrier_date = 1,783 missing values
   - order_delivered_customer_date = 2,965 missing values

   The order_status column has eight unique values, with “delivered” being the most frequent, appearing 96,478 times. This suggests most orders successfully completed the delivery process. This makes this dataset reliable for analyzing delivery times and order outcomes.

2. **items DataFrame (olist_order_items_dataset.csv)**
   The olist_order_items_dataset.csv contains 112,650 rows and 7 columns, which indicates that each row represents one item within an order. The number of rows suggests that orders may consist of multiple items. This is beneficial for our project in analyzing basket size and multi-item purchase patterns. The preview shows key attributes including order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, and freight_value. This confirms the dataset captures transactional details at the order-item level, which will be

useful for both clustering (customer purchasing behavior) and association rule mining (product co-occurrence).

Moreover, all columns are populated with no missing values, and the dataset primarily uses object for identifiers, int64 for order_item_id, and float64 for price and freight_value. This clean schema simplifies downstream transformations because there is no need for data type conversions or imputations at this stage. However, the order_item_id is mostly 1, showing most orders likely have one unique line item, but values extend up to 21, confirming the presence of bulk/multi-item orders. Prices range widely (0.85 to 6735), indicating variability in product value, and freight charges also vary significantly, affecting the total customer spend.

3. **products DataFrame (olist_products_dataset.csv)**
   The olist_products_dataset.csv dataset has 32,951 rows and 9 columns that contains information about all unique products available in the e-commerce platform. Despite not containing duplicate rows, the initial exploration revealed missing values in several columns:
   - product_category_name, product_name_lenght, product_description_lenght, and product_photos_qty = 610 missing values each
   - product_weight_g, product_length_cm, product_height_cm, and product_width_cm = 2 missing values each

   Additionally, the data types are consistent, with identifier and category columns as object and numeric dimensions stored as float64. The most important insight we can derive from this exploration is the missing product_category_name values because this directly affects the effectiveness of the dataset for the Apriori algorithms. This also tells us the need for imputation for this dataset.

4. **product_cat DataFrame (product_category_name_translation.csv)**
   The product_category_name_translation.csv dataset contains 71 rows and 2 columns named product_category_name and product_category_name_english. The head() results confirm that the dataset holds mapping information between Portuguese category names and their English translations. Info and data types also indicate that both columns are of type object (string) and that there are no missing values. All rows are unique and no duplicate rows exist.

5. **customers DataFrame (olist_customers_dataset.csv)**
   The olist_customers_dataset.csv dataset contains 99,441 rows and 5 columns, providing details about customer IDs, unique identifiers, and geographic information like zip code, city, and state. There are no missing values or duplicate rows, indicating good data integrity. Only the customer_zip_code_prefix column is stored as an integer type, while the remaining columns are objects.

Clustering Preprocessing

An important step in preprocessing for clustering algorithms like KMeans is feature scaling. This step ensures that all numerical features contribute equally when using distance-based

algorithms like KMeans or Hierarchical Clustering. This is important because these algorithms rely heavily on distance metrics (Euclidean distance), where variables with larger ranges can disproportionately influence cluster formation.

For our customer segmentation objective, we derived features representing customer purchase behavior, such as:

- num_orders - frequency of purchases
- total_price - total spending (outliers are capped at the 99th percentile)
- unique_categories - diversity of product categories the customer has purchased from
- avg_spend_per_order - spending pattern per transaction, and
- avg_basket_size - quantity average per order.

These different features can of course have naturally different scales and clustering could prioritize variables with higher numeric ranges. For example, total spending can be in hundreds while basket size can be a small integer. To solve this, we used StandardScaler from sklearn.preprocessing. This scaler transforms features to have a mean of 0 and a standard deviation of 1, standardizing each feature's contribution. After scaling, all five behavioral features contribute equally to the distance computation, making the clustering outcome more interpretable. More importantly, for algorithms like KMeans, scaling makes convergence easier and ensures the resulting clusters are not biased toward one dimension.

Association Rule Preprocessing

For market basket analysis, our data must be represented in a “basket” format. To achieve this, we first combined order item details with product metadata and English category translations. This ensured that each product purchased in an order had a consistent, human-readable name (product_name). If no English translation existed, the original Portuguese name was retained, preventing data loss. Next, using groupby and unstack(), we converted transaction data from a long format (multiple rows per order) to a wide format (one row per order, multiple item columns). Each cell initially contained counts of items in that order. Finally, counts were then converted into binary values using .gt(0).astype(int), where 1 denotes the presence of an item and 0 denotes absence. This step is important because Apriori works on presence/absence rather than item quantities. This preprocessing step allowed the Apriori algorithm to identify frequent itemsets and product affinity, and understand transaction-level purchasing behavior at scale in our dataset.

B. Customer Segmentation using Clustering

Research on Clustering Algorithms:

Task: Research at least two different clustering algorithms (e.g., KMeans, DBSCAN, Hierarchical Clustering).

KMeans is a partition-based clustering algorithm that assigns points to the nearest K centroids and iteratively refining centroid positions. Known for its simplicity and speed, KMeans form

compact and roughly spherical clusters by minimizing within-cluster variance (Idrees, 2024). It starts by choosing the number of cluster k using techniques like the elbow method or silhouette score. After initializing centroids, each data point's distance to each centroid is calculated, assigning them to the nearest centroid and forming initial clusters. Afterwards, for each cluster, the mean of all the points assigned to it is calculated and moved to this new mean position. Reassigning points and recalculating centroids are done iteratively until the centroids stop changing significantly or a maximum number of iterations are reached.

On the other hand, Density-Based Spatial Clustering of Applications with Noise or DBSCAN is a density-based algorithm that groups together points that are closely packed and labels points in sparse regions as noise. (Tate, 2023). Compared to KMeans clustering, DBSCAN does not require specifying the number of clusters beforehand. Instead, clusters emerge where data density is high, making DBSCAN well-suited for discovering arbitrarily shaped clusters and handling outliers.

Research Question

Discuss in your notebook the pros and cons of each, and justify why you chose a particular algorithm for your dataset.

KMeans Clustering

Pros

- **High Efficiency and Scalability**
    - KMeans runs in approximately linear time relative to the number of points, making it very fast on large datasets. For example, a retail study with ~542,000 customers found KMeans achieved the best clustering quality (highest silhouette score) among algorithms tested (Turkmen, 2022).
- **Handles Large Dataset with Moderate k**
    - KMeans does not require storing a large distance matrix, making it efficient in handling tens of thousands or more of data points as long as the number of clusters (K) is not extremely high (Tate, 2023). This made it suitable for our 90k dataset in terms of runtime.

Cons

- **Assumes Spherical Clusters of Equal Density**
    - According to Idrees (2024), KMeans implicitly biases toward equal-size, convex (sphere-like) clusters due to its use of centroids and Euclidean distance. Because of this, it may perform poorly if true clusters are irregularly shaped or vary greatly in size.
- **Sensitive to Outliers**
    - Tate (2023) emphasized that KMeans has no built-in notion of "noise" as even a few outlier points can drag the centroid towards them, distorting cluster assignments. In contrast, DBSCAN can label such points as noise.

DBSCAN Clustering

Pros

- **Find Arbitrary-Shaped Clusters**

    - BSCAN can detect clusters of any shape as it bases on connectivity of dense regions. This is ideal if customer segments do not form direct spheres in our feature space.
- **Robust to Noise and Outliers**
    - A key strength of DBSCAN is labeling outliers as noise points (Tate, 2023). Customers who don't fit well into any cluster can be isolated as noise, instead of forcing them into a cluster where they don't truly belong. This leads to more meaningful clusters especially when working with messy real-world data.

Cons

- **Struggles with Varying Densities**
    - In a customer dataset, if one segment is tightly clustered (e.g. very similar customers) and another is more diffuse, DBSCAN might either break the sparse cluster into noise or merge the dense cluster with adjacent points. This issue can cause DBSCAN to miss some clusters or combine others inappropriately. (Tate, 2023; Idrees, 2024).
- **Computations Cost on Large/High-Dimensional Data**
    - In worst-case scenarios, DBSCAN can have very high computational costs. For the 90k+ data points involved in our project, DBSCAN can still be feasible but it will be slower than KMeans.
- **Less Straightforward Cluster Summaries**
    - DBSCAN outputs clusters as sets of points without a centroid or explicit representative. This means after clustering, we might need an extra step to characterize each customer segment like computing the average profile or other descriptors for points in a cluster. In contrast, KMeans can directly provide centroid as segment prototypes.

Justification of KMeans for This Project

We selected KMeans as the clustering algorithm for our customer segmentation because it aligns well with our dataset and project goals. The dataset has over 90,000 records with scaled numerical features, which KMeans handles efficiently due to its linear-time complexity with respect to data size. This allowed us to perform segmentation quickly without significant computational overhead, making KMeans practical for exploratory segmentation and rapid iteration.

KMeans also provides centroids as cluster prototypes, which make each customer segment easy to interpret. These centroids represent the "average customer" in each segment, allowing us to describe customer groups by meaningful behaviors such as order frequency, average spending, and basket size.

One limitation of KMeans is its bias towards spherical, equally sized clusters. To solve this, we selected behaviorally meaningful and normalized features like the number of orders, total spend, unique product categories, and average basket size so that each dimension contributes equally to distance calculations. We also used feature scaling via StandardScaler and verified cluster quality using both the Elbow Method and the Silhouette Score, choosing a cluster count (k=3) that produced well-separated clusters despite potential variations in cluster shapes.

With regards to KMeans being sensitive to outliers, we addressed this by capping extreme spending values at the 99th percentile to reduce skew from rare, extreme purchases. Additionally, we derived metrics like average spend per order and average basket size further normalized customer behavior, reducing the impact of single anomalous purchases.

Model Building & Evaluation

Elbow Method

We used the Elbow Method and the Silhouette Score to determine the optimal number of clusters (k) for KMeans clustering.

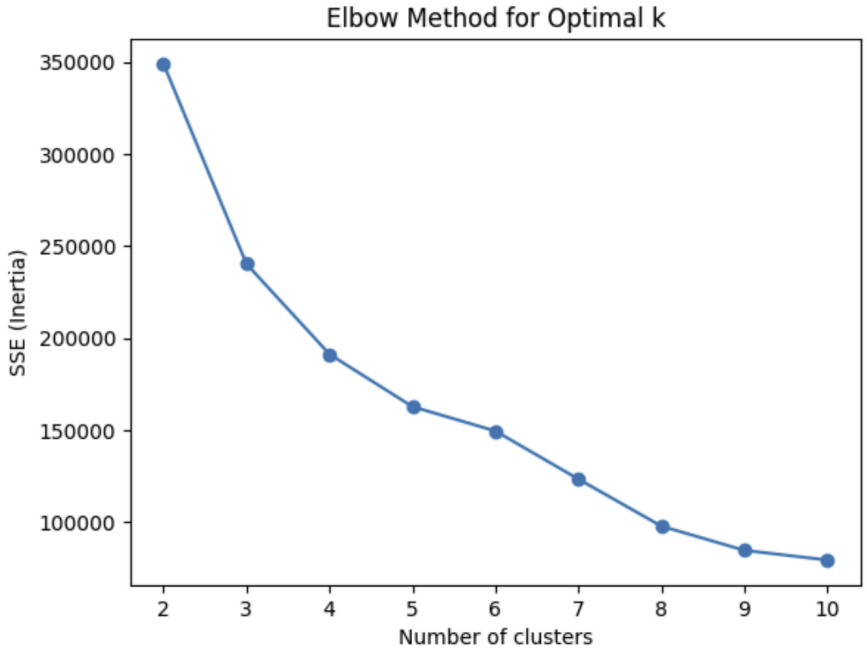


Figure 1. Elbow Method for Determining the Optimal Number of Clusters (k)

```
k=2, silhouette score=0.780
k=3, silhouette score=0.778
k=4, silhouette score=0.695
k=5, silhouette score=0.714
k=6, silhouette score=0.715
```

Figure 2. Silhouette Score Evaluation for k-Values

The Elbow Method involves plotting the Sum of Squared Errors (SSE) (or inertia) against different values of k. As the number of clusters increases, SSE naturally decreases because clusters become smaller and more specialized. However, beyond a certain point, the rate of improvement slows down, forming an “elbow” shape in the graph. This point typically represents an optimal balance between cluster compactness and model simplicity. From the elbow plot (see Figure 1), a

noticeable bend occurs around k=3 to k=4, indicating diminishing returns in SSE reduction beyond this range.

Moreover, the Silhouette Score measures cluster quality by considering both cohesion (how close points are within a cluster) and separation (how far clusters are from each other). Scores range from -1 to 1, with higher scores indicating better-defined clusters. For our dataset, k=2 scored the highest (0.780), but it provides only two broad customer segments, which may oversimplify business insights. However, k=3 achieved a strong score of 0.778, closely matching k=2 while providing more granular segmentation. Increasing k beyond 4 reduced silhouette scores, showing reduced cluster quality despite additional clusters (see Figure 2).

Considering both methods, we selected k=3 as the optimal number of clusters for our customer segmentation analysis.

Clustering Model Implementation and Label Assignment

With the optimal number of clusters determined as k = 3 from the Elbow Method and Silhouette Score analysis, the next step was to fit the KMeans clustering model on the scaled customer feature dataset. Using the standardized input matrix (X), the KMeans algorithm iteratively refined three centroid positions. It also assigned each customer to the closest centroid based on Euclidean distance. This approach ensured that customers with similar purchasing behaviors were grouped together.

The .fit_predict() method was applied, producing a cluster label (0, 1, or 2) for every customer in the dataset. These labels were then appended to the original customer features dataframe as a new column (cluster). This addition permanently linked each customer record to its respective segment and allowed us to later profile customers and visualize results.

Evaluation

Research Question

Explain why common supervised metrics (like accuracy) cannot be used for clustering evaluation.

The common supervised metrics, such as accuracy, precision, recall, and F1-score, are only possible under this machine learning technique because ground truth labels are present. These are benchmarks that we can directly compare to the predicted labels to measure the model's performance. Here in clustering, which is under the unsupervised learning technique, no predefined class labels are present. Thus, the common supervised metric will not be possible to use.

Since the algorithm discovers natural groupings in the data, we use the intrinsic metrics to measure how well it has captured the internal structure of the data, such as the Silhouette Score and the Davies-Bouldin Index (DBI). The Silhouette Score measures how similar a point is to its own group compared to the others, proving that it truly belongs to the group and that clusters are different and well-separated. In addition, the DBA measures the average similarities between clusters and other similar ones, proving that clusters are compact and distinct. Overall, both of them evaluate the

clusters depending on their compactness and separation, which are the best-suited metrics for the unsupervised technique.

Interpretation



Figure 3. PCA Visualization of Results

cluster	num_orders	total_price	unique_categories	total_items	avg_spend_per_order	avg_basket_size	pca1	pca2
0	1.012453	104.851336	1.000000	1.124590	103.687522	1.109999	-0.307383	-0.070764
1	1.025443	714.139965	1.004771	1.569514	834.100357	1.519272	4.505199	-1.947115
2	1.857626	234.065776	2.045205	2.553866	137.636650	1.522021	3.133874	6.271422

Figure 4. Aggregated Cluster Summary Table

The clustering analysis revealed three distinct customer segments, each with unique purchasing behaviors. The first segment (Cluster 0) consists of customers who make very few purchases, averaging only one order. Their spending per order is the lowest, at around $104, and their basket size is minimal, typically containing just one item. Additionally, these customers tend to purchase from only one product category. This group represents low-value, infrequent shoppers who may be more price-sensitive or only shop when they have a specific need. For this segment, the

retail company could focus on engagement campaigns, such as personalized discounts or loyalty programs, to encourage more frequent purchases and introduce them to a wider range of products.

The second segment (Cluster 1) is made up of customers who also purchase infrequently, averaging about one order, but they spend significantly more per transaction, with an average spend of around $834. Their basket size is larger than Cluster 0, and they still tend to focus on one product category. These are high-value, one-time buyers who spend a lot in a single transaction but do not return often. For these customers, the company could develop post-purchase engagement strategies, such as exclusive offers or targeted follow-ups, to increase customer retention and turn them into repeat buyers.

The third segment (Cluster 2) consists of customers who purchase more frequently, averaging nearly two orders. Although their average spend per order is moderate at about $138, they buy from more product categories and have the highest basket size. This group represents loyal, multi-category shoppers who demonstrate consistent purchasing behavior and broader product interest. The company can focus on upselling and cross-selling strategies for this group, offering loyalty perks and personalized recommendations to increase spending per transaction and further strengthen their relationship with the brand.

C. Market Basket Analysis using Association Rules

Research on Association Rule Algorithms

Discuss their fundamental differences and why Apriori is commonly used.

Apriori Algorithm

This is the algorithm used for finding groups of items in data and discovering useful patterns about how they are related (GeeksforGeeks, 2018b). This uses a horizontal format to store data and follows the breadth-first strategy. Furthermore, this uses the Apriori property that states that “if an itemset is infrequent, all its supersets must also be infrequent,” that helps prune the search space.

ECLAT (Equivalence Class Clustering and bottom-up Lattice Traversal) Algorithm

This algorithm is also used to find frequent itemsets in a dataset to create association rules (GeeksforGeeks, 2019b). Compared to Apriori, this uses a vertical format to store data and utilizes the depth-first search strategy with fewer scans.

Comparing the Association Rule Algorithms

Feature	Apriori	Eclat
Search	Breadth-first	Depth-first
Data Format	Horizontal format	Vertical format

Support Count	Multiple, entire database scan	Transaction ID set intersection
Efficiency	Slower if large datasets	Faster for practicality

Despite these differences, Apriori is still used commonly because it is **simple** and **easy** to understand and implement. The pruning mechanism of the Apriori property significantly reduces the itemsets to evaluate, which is incredibly beneficial. Because it is an old algorithm, it has been the classic and foundational algorithm for other newer algorithms, such as ECLAT.

Apriori Evaluation

The Apriori Metrics

Metric	**Description**
Support	Measures how frequently an itemset appears in the dataset
Confidence	Measures the likelihood of B being bought when A is bought
Lift	Measures the rule strength beyond chance

Evaluating the Market Basket Analysis of the Dataset

In our project, we see the same metrics that evaluate the transaction data. This has been visualized using a scatter plot for easier understanding.

On the X-axis, we can see the metric, support, wherein most rules are **below 0.06** which is **low**. This means that they apply to a small portion of all the transactions. This is possible as there is a very large number of possible item combinations, and most products are not often bought together. However, on the far right of the graph, a few rules have a higher support with a value **greater than 0.16**. This tells us that this rule happens more frequently in the data, which **indicates a common purchasing behavior**.

On the Y-axis, we can see another metric, confidence, where one point stands out with a **value of 1.0**. This tells us that the **consequent of this rule always appears when the antecedent is present**. However, many of the rules cluster along the values **ranging from 0.1 to 0.4**, which are low. This implies that **these combinations exist but does not always guarantee the presence of the consequent**.

Another metric that can be seen in the scatter plot is the bubble size and color of the rules. The most evident one is at the big red bubble at the top left of the graph with a value of **7.5**, which is the most evident as well with the confident metric. This indicates that this rule is

rare (low support) but strong (itemset occurs when present), suggesting a reliable and significant pattern. Other rules still have values **greater than 1.0 (ranging from 1.5 - 4.5)**, which indicates a **meaningful association**, with every greater value hinting stronger links.

Significance of Lift Value Greater Than 1

Why is a lift value greater than 1 significant in association rules?

A lift greater than one means a positive association. This means that whenever A is purchased, the likelihood of B being purchased as well is increased. This tells us that it is not solely by random chance that these two have been purchased together.

Other values, such as an exact 1, mean no real association between the itemset. While a lesser value than 1 indicates a negative correlation, wherein when A is bought B is less likely to be bought.

Visualization Interpretation

	antecedents	consequents	support	confidence	lift
0	health_beauty, computers_accessories	sports_leisure	0.002342	1.000000	8.211538
1	health_beauty	perfumery	0.028103	0.235294	4.186275
2	perfumery	health_beauty	0.028103	0.500000	4.186275
3	bed_bath_table, auto	cool_stuff	0.002342	0.500000	3.618644
4	electronics	computers_accessories	0.011710	0.263158	2.809211

Figure 5. Top Five (5) Rules Output

Rule 1	
Antecedent	health_beauty, computers_accessories
Consequent	sports_leisure
Support	0.23%
Confidence	100%
Lift	8.21

This tells us that every time customers buy both health & beauty items and computer accessories, they also buy something from sports & leisure.

This presents a strong opportunity to present a cross-category bundle. Despite being rare (low support), the consistency (100% confidence) and non-randomness (8.21 lift value) is good to create custom bundles or promotions that combine technology and health products. Promotions such as a purchase of beauty and gadget essentials such as Dyson Hairwrap would give a discount to a gym bag would help.

Rule 2	
Antecedent	health_beauty
Consequent	perfumery
Support	2.81%
Confidence	23.5%
Lift	4.19

This tells us that 1 in 4 customers that buy health & beauty products also buy perfumery items. Furthermore, they are over 4 times more likely to buy than random chance.

This presents cross-selling within the beauty category. This can be applicable for online store layouts, such as placing perfumes near the health & beauty products. Other bundles such as beauty kits that offer a fragrance bundle like body wash and perfume can also work.

Rule 3	
Antecedent	perfumery
Consequent	health_beauty
Support	2.81%
Confidence	50%
Lift	4.19

This tells us that half of the customers that buy perfumes also buy health & beauty products.

This is the reverse of the second rule above that suggests perfumes go alongside beauty care items. Promotions such as “Buy 2 Perfumes, Get Free Face Powder” could work alongside promoting a new product. This also reinforces the rule as said earlier.

Rule 4	
Antecedent	Bed_bath_table, auto
Consequent	cool_stuff
Support	0.23%
Confidence	50%
Lift	3.62

This tells us that half of the customers that buy home products and auto-related items buy from the cool stuff category, which are lifestyle items.

This is a very specific group that is useful for targeted campaigns or loyalty perks. Bundles such as “Shop for Home and Car Items and Unlock Exclusive Lifestyle Items” could work that applies to this rule.

Rule 5	
Antecedent	electronics
Consequent	computers_accessories
Support	1.17%
Confidence	26.3%
Lift	2.81

This tells us that around 1 in 4 electronic buyers also buy computer accessories. This is 2.8 times more likely than average.

This presents another opportunity to recommend new products to a customer. Promotions such as when a laptop is bought, freebies such as keyboards or wireless mouse are ideal for customers to buy.

D. Reinforcement Learning: Conceptual Demonstration

Research on Reinforcement Learning

Reinforcement Learning (RL) comes alongside Supervised and Unsupervised learning techniques, which mainly focus on agency through trial and error (GeeksforGeeks, 2018a). This machine learning occurs with an interaction with the environment to reach a goal, receiving feedback of either a reward or a penalty depending on their actions. Through the reinforced feedback, the machine learns over time.

Core Concepts of RL

Term	Description
Agent	Decision-maker performing actions to achieve a goal
Environment	Where the agent operates
State	Current situation of the agent
Action	Possible decisions of the agent
Reward	Feedback based on the action of the agent to guide the agent to a goal
Policy	Maps states into actions; Agent's strategy

Q-Learning

This is a reinforcement learning algorithm that learns through updating a table of values (named Q-table). Each entry in the Q-table represents the value of each action in a state (Kovalchuk, 2024). This is an off-policy algorithm that learns from the optimal action developed without the actual choices of the agent.

Q-Learning Process (GeeksforGeeks, 2019a)

1. **Starting with a state**
   In a gridworld environment, the agent starts with a state with its initial location, such as (3, 0). This location is also the agent's state.

2. **Agent selects an action**
   The agent navigates the grid to reach a goal, choosing a move in the grid based on its current state. The Q-table helps in guiding the agent with random choices for new actions or good actions from current Q-values.

3. **Action is done; Environment gives feedback**
   Once an action is done, a new state is given as the result (new location), and feedback is given based on the action (reward). In this example, all moves in the grid give no

reward. When the goal is reached, a reward of +10 is given. If a wall is encountered by the agent, a reward of -1 is given, which is a penalty.

4. **Update Q-table; Policy is refined**
   The Q-table is updated from its current run and adjusted based on the received reward and new state. The agent's strategy is refined for choosing an action in each state. This estimates which actions are more beneficial over time.

5. **Cycle repeats**
   The entire process repeats until the optimal policy is developed with the highest possible reward.

SARSA

SARSA (State-Action-Reward-State-Action) learns from its knowledge based on the actions the agent takes. In updating a rule, an immediate reward is received, then a future reward is expected. This is an on-policy algorithm that learns from the current policy followed and the actual actions of the agent (Amit, 2024).

SARSA Process

1. **Starting with a state**
   In a gridworld environment, the agent starts with a state with its initial location, such as (3, 0). This location is also the agent's state.

2. **Agent selects an action**
   The agent navigates the grid to reach a goal, choosing a move in the grid based on its policy.

3. **Action is done; Environment gives feedback**
   Once an action is done, a new state is given as the result (new location), and feedback is given based on the action (reward). In this example, all moves in the grid give no reward. When the goal is reached, a reward of +10 is given. If a wall is encountered by the agent, a reward of -1 is given, which is a penalty.

4. **Update State-Action Pair; Policy is refined**
   The state-action pair is updated using SARSA. A state and action are then set based on the received reward and the policy. This policy is not guaranteed to be the best possible action. The agent's strategy is refined for choosing an action in each state. This estimates which actions are more beneficial over time.

5. **Cycle repeats**
   The entire process repeats until the optimal policy is developed with the highest possible reward. Over many runs, the policy converges toward better decisions by continuously improving the Q-values based on actual experience.

Research Question

Explain how RL differs fundamentally from supervised and unsupervised learning. Why isn't it considered an unsupervised technique?

Comparing the Machine Learning Techniques

ML Technique	Description
Supervised	Learns from labelled data
Unsupervised	Finds patterns in unlabelled data
Reinforcement	Learns through interactions

RL is not considered an unsupervised technique as it uses states, actions, and iterations to learn in its environment. Unsupervised learning focuses on finding patterns through grouping or finding similarities between variables. While RL may learn patterns over time, this mainly learns through trial and error with rewards and penalties.

Scenario: Intelligent Shelf Restocking

Scenario Description: An agent learns to optimize shelf restocking
Problem: A store wants to improve how or when products are restocked throughout the day. Fixed schedules of restocking lead to lost sales, wasted space, and employee time.

RL Concept	Application
Agent	A smart stocking system. Decides which items to restock, how many, and when
Environment	Retail Store with shelf space, product type, time of day, customer traffic, and stock levels
State(s)	● Current stock levels ● Time of day ● Historical sales ● Day of week ● Customer Traffic
Action	● Restock a product ● Delay restocking and wait for data ● Prioritize categories of products
Reward	● +10 if shelves are well-stocked and a sale is made ● -5 if the product goes out of stock when customers want it ● -2 for overstocking an item that doesn't sell ● 0 for neutral events (e.g., maintaining a safe stock level)

Policy	Based on states, choose the restocking action that keep shelves stocked just enough to meet demand without overstocking.

Conceptual Walkthrough

RL Conceptual Walkthrough

1. **State Space (S):**
   The state represents the current condition of the store, such as:
   - Stock levels (e.g., 2 bottles of water left)
   - Time of day (morning, noon, etc.)
   - Day of week (e.g., Monday)
   - Customer traffic (e.g., high, low)
   - Historical sales (e.g., rolling average)

   Example:
   S = (stock_levels = {'water': 2, 'chips': 5}, time = 11:30 AM, day = Monday, traffic = High)

2. **Action Space (A):**
   The agent can take actions like:
   - Restocking a specific product by a certain amount
   - Delaying restocking
   - Prioritizing product categories

   Examples:
   A = Restock('water', 5)
   A = Delay
   A = Restock('snacks', 10)

3. **Reward Function (R):**
   - +10: shelves are stocked and a sale occurs
   - -5: item is out of stock when needed
   - -2: overstocking an item that doesn't sell
   - 0: safe but neutral stock level (no sale or loss)

RL Learning Process

1. **Starting with a state**
   Agent begins in a known state:

   S = (stock_levels = {'water': 2, 'chips': 5}, time = 11:30 AM, day = Monday, traffic = High)

   This will serve as the current condition of the environment and basis for the next decision of the agent.

2. **Agent selects an action**
   Based on the current state, the agent selects an action such as restocking a specific item, delaying restocking, or prioritizing a category. Initially, choices may be exploratory (random), but over time, they become guided by the learned policy.

3. **Action is done**
   Based on the current state, the agent chooses an action. Actions may be random at first, but over time, actions are chosen based on the agent's policy. These actions could include restocking a specific product by a certain amount, delaying restocking, or prioritizing on a specific category like beverages or snacks.

4. **Environment gives feedback**
   Once the action is done, say, the restocking of 5 bottles of water, the environment changes. Customers play a part by making a purchase or not; with this, the agent receives feedback based on the reward function.

5. **Update state-action value; Policy is refined**
   The agent updates its state-action pair based on the reward and new state. Multiple runs make the agent learn which actions are optimal. The policy is, then, refined to choose the higher reward over time. Examples of these learned actions and refined policies are restocking fast-selling items before peak hours or delaying low-priority items during low traffic.

6. **Cycle repeats**
   The entire process repeats until the optimal policy is developed with the highest possible reward. Over many runs, an optimal policy is developed that balances customer demand, product availability, and shelf space usage, leading to improved store performance.

Discussion

Real-World Retail Setting: Benefits

1. **Dynamic Decision-Making**
   RL adapts to changing conditions. In the restocking scenario, the agent adjusts its decision based on real-time factors, which leads to smarter restocking compared to relying on fixed restocking schedules. This can even be helpful despite changing customer behavior or other state changes.

2. **Improved Inventory Management and Operational Efficiency**
   In the example, RL reduces the loss of sales, wasted space, and employee time. For example, an agent might learn that demand for bottled water spikes before lunch on weekdays and prioritize restocking then, which optimizes the store's inventory.

In turn, employees can now focus on customer service. Over time, the agent learns patterns that humans might overlook, such as subtle seasonal demand shifts, improving long-term efficiency.

3. **Scalability and Personalization**
   Once trained, an RL system can be deployed across multiple branches with similar layouts or inventory. It can also adapt to location-specific behaviors, such as weekend shopping surges in urban areas versus rural ones.

Real-World Retail Setting: Challenges

1. **Complexity of Environment**
   The real world is noisy, unpredictable, and sometimes more unique than we thought. Certain scenarios may occur that cannot be accounted for by the RL, and the learned policy may overfit to patterns that don't generalize well.

2. **Initial Data Problems**
   Starting may be a challenge for the RL as it needs interaction for effective learning. In our shelf restocking case, the agent would need to experiment with many restocking decisions and observe outcomes over time. Early in training, poor decisions could be bothersome to train in simulation, or badly affect sales and customer experience in the real world.

3. **Reward Design Complexity**
   A reward function is also difficult to implement, especially with the business rules. For the RL to be given a reward only when sales occur might overlook long-term customer satisfaction or brand loyalty, which are hard to quantify in a simple reward function. A poorly designed reward can lead to unintended behaviors like prioritizing only profitable items regardless of customer needs.

4. **Integration with Existing Systems**
   An RL in a real-world retail store may be costly, as it should be designed to support real-time decision-making. Furthermore, it would need integration with the inventory, POS, and employee schedules.

5. **Safety and Risk Management**
   Considering the early poor decisions of the agent, learning in the real world could possibly hurt the business or the customers themselves.

E. Overall Discussion

Synthesis

In this project, we segmented over 90,000 retail customers based on their purchasing behavior using the KMeans clustering algorithms. After preprocessing the data, we used the

Elbow Method and the Silhouette Score to identify the optimal number of clusters k = 3. The resulting clusters revealed three distinct customer segments: low-spending, infrequent shoppers who make small, single-category purchases (Cluster 0); high-spending, infrequent buyers who spend significantly more in single transactions (Cluster 1); and more loyal, frequent shoppers who purchase across multiple product categories and maintain larger basket sizes (Cluster 2). Through these insights, we established the need for the retail company to adopt targeted marketing strategies, such as loyalty programs for Cluster 0, retention-focused campaigns for Cluster 1, and upselling or cross-selling for Cluster 2, improving customer engagement and overall business performance.

For the association rule mining analysis, this revealed several actionable patterns in the purchasing behavior of customers that the business can utilize for strategy. Despite most of the discovered rules having support, these showed high confidence and lift. This tells us they specifically apply to transactions but are still strong and not random among product categories. The most evident rule here is the health & beauty, computer accessories, and sports & leisure that stood out with a perfect confidence and a lift of 8.21. This is small but still consistent with a customer segment with specific interest, suggesting that they can be targeted with promotions or bundles. On the other hand, the other rules still showed strong relevant relationships such as health & beauty and perfumery, which support strategies like shared promotion or placing items together. Overall, this highlighted opportunities to customize marketing according to a targeted customer group, optimize online store layouts, and create product bundles that mirror real-world buying patterns, even in less frequent but high-value itemsets.

Research Question

How do the insights from clustering complement those from association rules?

Clustering and association rule mining offer different perspectives on customer behavior, which is why they complement each other. Clustering segments based on customers are based on behavioral features such as the purchase frequency, total spending, and the variety of products. This helps us understand who the customers are and their habits. For example, our clustering analysis identified three (3) segments: low-engagement shoppers who place small and one-time orders, high spending but infrequent buyers, and loyal customers who order in multiple categories with the largest basket sizes. While association rules focus on the products typically bought together that give ideas for bundles, promotions, and store layouting (even if online). Together, **they complement each other by allowing them to target the right customer segments with the right product combinations**, leading to more personalized marketing and improved customer experience.

F. Conclusion

In conclusion, this project successfully met its objective of segmenting a large customer base into meaningful groups using clustering techniques. By preprocessing and aggregating customer transaction data, scaling numerical features, and applying the KMeans algorithm, we identified three distinct customer segments with unique purchasing behaviors. These segments

revealed actionable insights: low-spending occasional shoppers, high-value but infrequent buyers, and loyal customers with diverse purchasing patterns and larger basket sizes. Determining the optimal number of clusters through the Elbow Method and Silhouette Scores ensured that the model achieved a balance between accuracy and interpretability.

As for the Association Rule Mining, we have identified relationships between product categories that reveal opportunities for cross-seeling, bundling products, and even product placement. While Reinforcement Learning showcased smart restocking that adapts to dynamic customer behavior and store conditions. Overall, these techniques have met our project objectives through applying the unsupervised machine learning techniques and providing both descriptive and predictive insights that can possibly improve sales, service, and customer experience of the Brazilian E-Commerce, Olist.

G. Recommendations

Customer Segments

- For Low-Spending Occasional Shoppers:
    - Launch personalized discounts and introductory offers to encourage repeat purchases.
    - Promote loyalty programs or point-based rewards to build engagement.
    - Highlight affordable and trending products to appeal to their price-sensitive behavior.
- For High-Value but Infrequent Buyers
    - Offer exclusive promotions, such as VIP memberships or early access to premium products.
    - Implement post-purchase follow-ups to improve retention.
    - Target these customers with upselling campaigns to increase their order frequency.
- For Loyal and Diverse Shoppers
    - Provide personalized product recommendations and bundle deals to boost average basket size.
    - Reward multi-category purchasing with exclusive perks or loyalty bonuses.
    - Maintain engagement through appreciation campaigns like “preferred customer” status or seasonal rewards.

Association Rules

- Implement integrated bundle deals across different product categories
- Boost the sales of related items within the beauty category
- Feature electronic product recommendations during checkout
- Offer exclusive rewards to specific customer segments
- Strategically organize online store layout the by grouping related products together

Reinforcement Learning

- Utilize RL for dynamic and real-time restocking decisions

- Simulate RL training in a virtual environment before real-world deployment to avoid costly budgets
- Design a well-balanced reward function that reflects both immediate and long-term business goals
- Utilize RL data to improve the efficiency and effectiveness of manual operations.
- Plan gradual system integration with store platforms used for RL to be fully utilized later
- Customize policies for different store branches to determine patterns in each location

References

Amit, H. (2024, December 2). *SARSA Algorithms Explained - Hey Amit - Medium*. Medium. https://medium.com/@heyamit10/sarsa-algorithms-explained-b94078fd658b

GeeksforGeeks. (2018a, April 25). *Reinforcement Learning*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/

GeeksforGeeks. (2018b, September 4). *Apriori Algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/apriori-algorithm/

GeeksforGeeks. (2018c, September 13). *Association Rule*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/association-rule/

GeeksforGeeks. (2019a, February 6). *QLearning in Reinforcement Learning*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/

GeeksforGeeks. (2019b, June 11). *ECLAT Algorithm ML*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/ml-eclat-algorithm/

GeeksforGeeks. (2019c, June 12). *Implementing Apriori algorithm in Python*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/implementing-apriori-algorithm-in-python/

Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques. In *Data Mining* (3rd ed., pp. 243–278). Morgan Kaufmann. https://doi.org/10.1016/b978-0-12-381479-1.00006-x

Idrees, H. (2024, October 22). K-Means vs. DBSCAN: Clustering Algorithms for Grouping Data. Medium. Retrieved July 28, 2025, from https://medium.com/@hassaanidrees7/k-means-vs-dbscan-clustering-algorithms-for-grouping-data-a4969034cfcc

Kovalchuk, G. (2024, October 24). *A Beginner's Guide to Q-Learning: Understanding with a Simple Gridworld Example*. Medium. https://medium.com/@goldengrisha/a-beginners-guide-to-q-learning-understanding-with-a-simple-gridworld-example-2b6736e7e2c9

Raschka, S. (2014). *mlxtend*. Rasbt.github.io. https://rasbt.github.io/mlxtend/

Tate, A. (2023, October 24). When To Choose Density-Based Methods. Hex. https://hex.tech/blog/comparing-density-based-methods/#:~:text=DBSCAN%20is%20a%20density,the%20density%20of%20data%20points

Turkmen, B. (2022). Customer Segmentation With Machine Learning for Online Retail Industry. The European Journal of Social & Behavioural Sciences, Volume 31(Issue 2), 111-136. https://doi.org/10.15405/ejsbs.316